

Machine learning applications for Rothe’s method

Brage Brevig^{a)} and Simon Elias Schrader^{b)}

(Dated: 5 June 2025)

Rothe’s method is a relatively new method to solve the time-dependent Schrödinger equation (TDSE). The main bottleneck in Rothe’s method is that a very demanding optimization problem has to be solved at every time step, requiring hundreds of iterations. To tackle this problem, we used two types of machine learning approaches - Learning to optimize (L2O) and graph convolutional networks for time series prediction, to carry out the optimization itself using neural networks and to learn good initial guesses for the optimization problem, respectively. For Rothe’s method, we observe that the learned optimizer can outperform the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm in problems that resemble the training problems, showing that L2O has the potential to speed up calculations with Rothe’s method dramatically. We observe that an autoregressive ansatz to obtain an initial guess for the parameters at the next iteration can be implemented effectively using either a long short-term memory neural network (LSTM) or a time graph convolutional network (t-GCN), outperforming heuristic ways to predict good initial parameters. These two applications of machine learning are another step towards making Rothe’s method competitive to existing methods for solving the TDSE.

I. INTRODUCTION

Solving the TDSE is a numerically extremely demanding task. To solve the TDSE exactly for N particles, a naive implementation requires exponentially many $3N$ -dimensional basis functions (e.g. Slater Determinants), which corresponds to the FCI method. In addition, for atoms and molecules in strong fields, charged particles move far away from their initial positions. This further increases the number of basis functions necessary per particle when a product basis is used. While there exist many methods that try to overcome this problem, such as the multiconfigurational time-dependent Hartree-Fock method or time-dependent Coupled Cluster methods,¹⁻⁴ the computational scaling is still very steep due to the large grids required. The standard basis sets of quantum chemistry, Gaussian basis sets, are not applicable, as these basis functions are localized close to the nuclei, and the development of Gaussian basis sets for strong-field quantum dynamics is still an ongoing field of research.⁵⁻⁸ An alternative is to use *time dependent* Gaussian basis sets, where the underlying Gaussians themselves change over time. Recently, Rothe’s method has been applied to time-dependent Gaussians, where the wave function (or the orbitals) are written as a linear combination of time-dependent Gaussian functions.⁹⁻¹¹ In Rothe’s method, the time evolution is rephrased as an optimization problem for the nonlinear parameters of each Gaussian (e.g. the width and position) at the next time step. Rothe’s method is numerically very demanding, however, as one has to solve a highly nonlinear optimization problem at every time step. Furthermore, it is not clear what one should use as the initial parameters for the optimization – bad initial parameters can lead to much slower convergence. Coming up with better initial parameters and reducing the number of optimization steps can hence speed up Rothe calculations.

L2O is a machine-learning application, where one *learns* the optimizer itself. That is, the update scheme to get from the parameters at epoch/iteration i to the parameters at epoch $i + 1$ is learned. L2O has recent applications in both quantum computing and in machine learning, and learned optimizers can perform on par with, or even outperform, the best analytical algorithms.¹²⁻¹⁸ This is because they can learn the underlying structure of the problem, and thereby outperform very general optimization algorithms that are agnostic to the problem structure – at the cost of no longer fulfilling analytical convergence properties and only being applicable to the type of problems they were trained on. Because each iteration of an optimization problem can be interpreted as a time step, the natural machine learning architecture to use here are recurrent neural networks (RNNs) and architectures extending RNNs, such as LSTMs.¹⁹

Recently, *neural* state space models (n-SSMs) have emerged as a contender to RNNs due to their numerical stability and ability to capture long-range dependencies in time-series.²⁰⁻²² Stemming from control theory and classical dynamics, continuous state space models are simple parametric representations for such systems, recurrently propagating their dependent variables forwards in time. We propose that optimization problems, as considered in this project, may be recast as dynamic systems initialized by some guess towards an optimum and propagated towards the correct solution by learning the transition matrices controlling the system via neural networks.

Graph Convolutional Networks (GCNs) can be considered a "generalization" of Convolutional Neural Networks (CNNs) and are applicable to structured data, where the relations between variables can be very general – there only needs to be an underlying graph structure.²³ GCNs can be applied to time series as well:²⁴ Here, the GCNs are responsible for the correlation between the input data, while an RNN is used to capture the time dependency.

In this paper, we apply a specific type of L2O-LSTM¹² to high-dimensional quadratic functions as a general test,

^{a)} Electronic mail: bragebr@uio.no

^{b)} Electronic mail: s.e.schrader@kjemi.uio.no

and to Rothe's method for the Hydrogen atom, where relevant functions have been implemented by one of the authors. We also investigate whether nSSMs are suitable for L2O as they in theory promise improved temporal- and computational complexity, as well as stability, compared to those of RNN-based networks. The highly flexible structures of these models are also suitable for scenarios for which added complexity in form, not in size, is necessary. We use a time-series GCN to evaluate whether we can predict the Gaussian parameters at the next time step based on the previous steps – the predicted value would then be able to serve as an excellent initial guess. This is implemented for the Henon-Heiles potential, where the relevant Gaussian parameters are available.

This paper is structured as follows: In section II, we introduce the relevant theory for this paper: Rothe's method, RNNs, LSTMs, classical optimization, L2O, nSSMs, and GCNs for time series. In section III, we describe in detail the cases we apply L2O and GCNs to - quadratic functions, Rothe's method for the Hydrogen atom, and time series data from Henon-Heiles calculations. In section V, we present our results and assess them critically. We summarize and contextualize our results in the **conclusion**.

II. THEORY

A. Rothe's method with Gaussian basis functions

Rothe's method is a numerical method to solve the TDSE. Starting from the TDSE in natural units ($\hbar = 1$),

$$i\frac{\partial}{\partial t}\Psi(t) = \hat{H}(t)\Psi(t), \quad (\text{II.1})$$

one can apply the Crank-Nicolson (CN) propagator using a time step h , an implicit method to solve ODEs, with second-order accuracy, i.e. a global error that goes as $O(h^2)$. Applying the CN propagator to propagate from time step t_i to timestep $t_{i+1} = t_i + h$, we get

$$\Psi(t_{i+1}) = \hat{A}_i^{-1} \hat{A}_i^\dagger \Psi(t_i), \quad (\text{II.2})$$

where

$$\hat{A}_i = \hat{I} + i\frac{h}{2}\hat{H}\left(t_i + \frac{h}{2}\right). \quad (\text{II.3})$$

A variational reformulation reads

$$\Psi(t_{i+1}) = \arg \min_{\chi} \left\| \hat{A}_i \chi - \hat{A}_i^\dagger \Psi(t_i) \right\|^2. \quad (\text{II.4})$$

This is Rothe's method.^{9,25} Each χ is now expanded in a basis of functions that parametrically depend on both linear and nonlinear parameters, i.e.

$$\chi(\theta(t), c(t)) = \sum_{m=1}^{M(t)} c_m(t) g_m(\theta(t)), \quad (\text{II.5})$$

where $M(t)$ is the number of basis functions, $c_m(t)$ is a set of linear coefficients and $\theta(t)$ is a set of nonlinear parameters - in principle, those can be shared among several basis functions, or one set of nonlinear parameters $\theta = \{\theta_m\}_{m=1}^{M(t)}$ can be associated with each basis function g_m . Introducing the *Rothe error*,

$$r_{i+1}(\theta, c) = \left\| \sum_{m=1}^{M(t)} c_m \hat{A}_i g_m(\theta) - \hat{A}_i^\dagger \Psi(t_i) \right\|^2 \quad (\text{II.6})$$

and the optimal parameters

$$\theta_{i+1}^{\text{opt.}}, c_{i+1}^{\text{opt.}} = \arg \min_{\theta, c} r_{i+1}(\theta, c), \quad (\text{II.7})$$

we see that eq. (II.4) is approximately solved for the function

$$\Psi(t_{i+1}) \approx \chi(\theta_{i+1}^{\text{opt.}}, c_{i+1}^{\text{opt.}}). \quad (\text{II.8})$$

By making use of the Variable Projection (VarPro) algorithm,²⁶ the optimal linear coefficients c for a given set of nonlinear parameters θ can be obtained analytically. Thus, the Rothe error becomes a function of the nonlinear parameters only,

$$r_{i+1}(\theta) = \left\| \sum_{m=1}^{M(t)} c_m(\theta) \hat{A}_i g_m(\theta) - \hat{A}_i^\dagger \Psi(t_i) \right\|^2. \quad (\text{II.9})$$

A strength of Rothe's method is that the number of basis functions used, $M(t)$, becomes explicitly a function of time. How $M(t)$ is determined, and several analytical properties of Rothe's method, is described in detail in Ref. 10.

In this paper, we consider two types of Hamiltonians and two types of systems. The first system we consider is the Hydrogen atom in a regularized Coulomb potential where the Hamiltonian reads, in atomic units

$$\hat{H}(t) = -\frac{1}{2}\nabla^2 - \frac{\text{erf}(100r)}{r} + E_0 f(t) \sin(\omega t) z. \quad (\text{II.10})$$

where $f(t)$ is a squared sine envelope,²⁷

$$f(t) = \begin{cases} \sin^2\left(\frac{\pi t}{t_f}\right), & \text{if } 0 \leq t \leq t_f, \\ 0, & \text{otherwise} \end{cases} \quad (\text{II.11})$$

where $t_f = 2\pi N_c/\omega$ and N_c is the number of optical cycles. For all simulations, $N_c = 3$ and $\omega = 0.057$ (a.u.). We consider different values of E_0 (see the methods section III). As basis functions, we use complex-valued, isotropic Gaussians, i.e.,

$$g_m(r, t) = g(r; \theta_m(t)) = d_m \exp\left(- (a_m^2 + ib_m) \|r - \mu_m\|^2 + ip_m^T(r - \mu_m)\right), \quad (\text{II.12})$$

where $\mu_m = (0, 0, \mu_m)^T$ and $p_m = (0, 0, p_m)^T$, and d_m is the unique positive real normalization constant. More computational details can be found in Ref. 10.

The second system we consider is the Henon-Heiles potential, which is used as a test system for vibrational dynamics, applicable to an arbitrary dimension D . The Hamiltonian reads in its natural units

$$\hat{H} = -\frac{1}{2} \sum_{i=1}^D \frac{\partial^2}{\partial r_i^2} + \frac{1}{2} \sum_{i=1}^D r_i^2 + \lambda_A \sum_{i=1}^{D-1} \left(r_i^2 r_{i+1} - \frac{1}{3} r_{i+1}^3 \right), \quad (\text{II.13})$$

where we set the strength of the anharmonicity $\lambda_A = 0.111803 \approx 1/\sqrt{80}$. The initial state is not an eigenstate, but a Gaussian with standard width displaced from the origin by 2 in all dimensions. That is,

$$\Psi(r, t=0) = \pi^{-D/4} \exp \left(- (r - \mu^0)^T A^0 (r - \mu^0) \right), \quad (\text{II.14})$$

where $A^0 = \frac{1}{2} I_D$ and $\mu_i^0 = 2$ for $i = 1, \dots, D$.

As basis functions, we use D -dimensional Explicitly Correlated Gaussians (ECGs). A D -dimensional ECG is a wave function of the form

$$g_m = g(\theta_m) = d_m \exp \left(- (r - \mu_m)^T A_m (r - \mu_m) \right), \quad (\text{II.15})$$

where A_m is a complex-symmetric $D \times D$ matrix with positive definite real part, $\text{Re}(A_m) > 0$ guaranteeing square-integrability, and μ_m is a complex vector of length D . One can generally parameterize A_m as

$$A_m = L_m L_m^T + i(K_m + K_m^T), \quad (\text{II.16})$$

where both L_m and K_m are real, lower triangular matrices. We parameterize μ as $\mu = q + ip$, where q and p are real vectors of length D . In this simulation, we tried to keep the number of Gaussians M constant. However, the Hamiltonian $\hat{H}$ is unbounded. Thus, part of the wave function will "leave" the region close to $r = 0$. As such, our implementation of Rothe's method for this potential had a way to remove Gaussians (but not to add Gaussians). For the models considered in this paper, we hence use a dataset of linear and nonlinear parameters starting at a time $t_0 > 0$, from which on no more Gaussians were removed. That is, for the purpose of this paper, the simulations are done with a constant number of Gaussians, with an initial state that is a linear combination of M Gaussians starting at $t_0 > 0$. We consider two datasets, one with $D = 2$ and $M = 29$ Gaussians and one with $D = 3$ and $M = 28$ Gaussians. For more details about the Henon-Heiles potential applied to Rothe's method, we refer to Ref. 11.

B. Recurrent Neural Networks (RNNs)

In this section, we will describe multi-layer RNNs. We consider a type of RNNs where there is just one weight

matrix $W_h^{(i)}$ per recurrent layer, and similarly, only one weight matrix between the previous layer $W_x^{(i)}$ and the actual layer, and assume no non-recurrent layers (even though it is in principle possible to put Multi-Layer Perceptron (MLP) Layers between the recurrent layers, and on both the input and the output). We use the general notation h_t to symbolize hidden recurrent layers, while $h_t^{(i)}$ is one particular recurrent layer.

RNNs are a type of neural network that are well suited for data with a temporal dependence, where we use the term "temporal" in the sense that the output at index $t+1$ depends, explicitly or implicitly, on the input at indices $t, t-1, \dots$. We will refer to these indices as *time*. The working equations for an L -layer RNN used for continuous-variable regression RNN read:

$$h_t^{(l)} = a_h^{(l)} \left(W_h^{(l)} h_{t-1}^{(l)} + W_x^{(l)} h_t^{(l-1)} + b_h^{(l)} \right) \quad (\text{II.17})$$

$$y_t = W_y h_t^{(L)} + b_y$$

where:

- The variable l stands for the layer, $l = 1, \dots, L$,
- $h_t^0 = x_t$ is the input at time t ,
- $b_h^{(l)}$, $l = 1, \dots, L$ is the bias of layer l ,
- b_y is the output bias,
- $W_h^{(l)}$ and $W_x^{(l)}$, $l = 1, \dots, L$, are weight matrices,
- $h_0^{(l)}$ is the initial hidden layer,
- $a_h^{(l)}$ is a nonlinear activation function for layer l , $l = 1, \dots, L$,
- y_t is the output at time t .

We consider all biases $b = \{b_h^{(l)}, b_y\}$, all weight matrices $W = \{W_x^{(l)}, W_h^{(l)}, W_y\}$, and the hidden states h_0 , to be learnable parameters.

While sufficiently large RNNs, in principle, can model any arbitrary time series and arbitrary long-term dependencies, they are hard to train: The default training algorithm, *backpropagation through time*, suffers from exploding/vanishing gradients, which makes RNNs de facto unable to handle long-term dependencies. In the next section, we describe an architecture that alleviates some of those problems.

C. Long short-term memory (LSTM)

LSTMs are a type of RNN, where each recurrent layer l is associated with a time-dependent hidden state $h_t^{(l)}$, in addition to a *memory cell* $c_t^{(l)}$. LSTMs have different

gates, so called *input gates* i , *forget gates* f , and *output gates* o . The equations characterizing an LSTM read

$$\begin{aligned}
i_t^{(l)} &= \sigma(W_i^{(l)} h_t^{(l-1)} + U_i^{(l)} h_{t-1}^{(l)} + b_i^{(l)}) \\
f_t^{(l)} &= \sigma(W_f^{(l)} h_t^{(l-1)} + U_f^{(l)} h_{t-1}^{(l)} + b_f^{(l)}) \\
o_t^{(l)} &= \sigma(W_o^{(l)} h_t^{(l-1)} + U_o^{(l)} h_{t-1}^{(l)} + b_o^{(l)}) \\
\tilde{c}_t^{(l)} &= \tanh(W_c^{(l)} h_t^{(l-1)} + U_c^{(l)} h_{t-1}^{(l)} + b_c^{(l)}) \quad (\text{II.18}) \\
c_t^{(l)} &= f_t^{(l)} \odot c_{t-1}^{(l)} + i_t^{(l)} \odot \tilde{c}_t^{(l)} \\
h_t^{(l)} &= o_t^{(l)} \odot \tanh(c_t^{(l)}) \\
y_t &= W_y h_t^{(L)} + b_y.
\end{aligned}$$

For LSTMs, the chosen activation functions are quite well-established as written, where σ stands for the sigmoid function. In comparison to an RNN, there are hence four times the number of weight matrices and biases per layer, with two weight matrices U and W and one bias b associated with the input gate, the forget gate, the output gate and the cell state. Each of these parameters is trainable, as are $h_0^{(l)}$ and $c_0^{(l)}$. Because $c_t^{(l)}$ can carry gradients across multiple time steps via

$$\frac{\partial c_t^{(l)}}{\partial c_{t-1}^{(l)}} = f_t^{(l)},$$

LSTMs can effectively maintain information over long sequences. Backpropagation through time works much better for LSTMs, and LSTMs are hence the preferred architecture for time series.

D. Classical optimizers

In optimization, the goal is to global minima of a function. Given a function $f(\theta)$ and perhaps its gradient $\nabla f_\theta(\theta)$ if it is differentiable, find one $\theta^* \in \Theta^*$ such that

$$\Theta^* = \arg \min_{\theta} f(\theta), \quad (\text{II.19})$$

where Θ^* is the set of all global minima, and θ^* one particular global minimum. For most functions, there is no closed-form solution for Θ^* , so iterative algorithms have been developed that minimize a function $f(\theta)$. Because saddle points, local minima (and local maxima) all have the property that $\nabla_\theta f(\theta) = 0$ at these points, most optimization algorithms are not able to find a global minimum - however, often, sufficiently small local minima or even saddle points are acceptable for a task in question.

A classical optimizer that does not use second-order information is an optimizer of the form

$$\begin{aligned}
\theta_{t+1} &= \theta_t + m_1(\nabla_\theta f(\theta_t), h_t) \\
h_{t+1} &= m_2(\nabla_\theta f(\theta_t), h_t). \quad (\text{II.20})
\end{aligned}$$

Here, we define the algorithm that optimizes $f(\theta)$ as the *optimizer*, while the function that is to be optimized is the *optimizee*. The simplest case is the gradient descent algorithm, where

$$m_1(\nabla_\theta f(\theta_t), h_t) = -\lambda \nabla_\theta f(\theta_t), \quad (\text{II.21})$$

and

$$m_2(\nabla_\theta f(\theta_t), h_t) = 0, \quad (\text{II.22})$$

where λ is the learning rate. More advanced methods construct one or several "inner states" $h_t = (h_{1,t}, h_{2,t}, \dots)$ that keep track about information from previous iterations and that are used in the construction of the update m_1 . For example, in the ADAM algorithm,²⁸ we have that

$$h_{1,t} = \beta_1 h_{1,t-1} + (1 - \beta_1) \nabla_\theta f(\theta_t), \quad (\text{II.23})$$

$$h_{2,t} = \beta_2 h_{2,t-1} + (1 - \beta_2) \nabla_\theta f(\theta_t) \odot \nabla_\theta f(\theta_t), \quad (\text{II.24})$$

$$\theta_{t+1} = \theta_t - \frac{\lambda}{\sqrt{h_{2,t}/(1 - \beta_2^t)} + \epsilon} \odot \frac{h_{1,t}}{1 - \beta_1^t}. \quad (\text{II.25})$$

For ADAM, it is customary to set $h_{1,t=0} = h_{2,t=0} = 0$, and $\lambda, \beta_1, \beta_2$ and ϵ are user-defined constants. Throughout this manuscript, we will use the default values as they are also defined in PyTorch:²⁹ $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$. What most classical optimizers have in common, is that they usually contain just a few parameters (in the case of gradient descent, there is just one parameter λ , in the case of ADAM, there are four parameters, $\lambda, \beta_1, \beta_2$, and ϵ) which can be tuned by the user and tailored to a specific problem. Methods like gradient descent and ADAM are developed to be applicable to a wide variety of problems, and good default values or ranges for the parameters $\beta_1, \beta_2, \lambda$ and ϵ have been developed. Nevertheless, these methods don't always perform equally well on all problems, and being general algorithms, they are not necessarily the "optimal" algorithm for a given class of problems.

E. Learning to optimize (L2O)

The main idea of (L2O) is that the functions m_1 and m_2 (eq. (II.20)) are not designed, but learned from data, giving rise to a learned optimizer. Making the functions m_1 and m_2 parametrically dependent on a large number of parameters ϕ , the general idea of L2O is to find the optimal parameters ϕ^* by minimizing a loss function

$$\mathcal{L}(\phi) = \mathbb{E}_{f \in \mathcal{F}} \left[\sum_{t=1}^T w_t f(\theta_t) \right] \quad (\text{II.26})$$

$$\text{where } \theta_{t+1} = \theta_t + g_t, \begin{bmatrix} g_t \\ h_{t+1} \end{bmatrix} = \begin{bmatrix} m_1(\nabla_\theta f(\theta_t), h_t, \phi) \\ m_2(\nabla_\theta f(\theta_t), h_t, \phi) \end{bmatrix}. \quad (\text{II.27})$$

Here, w_t are user-defined weights, and T is the *unrolling depth*. The expectation value $\mathbb{E}_{f \in \mathcal{F}}$ is taken over all functions of a class of functions $\mathcal{F}$ that we want to train the model on. More generally, the hope is that the learned optimizer performs well also on functions of a broader class of functions $\mathcal{G}$, where $\mathcal{F} \subset \mathcal{G}$. In principle, we are only interested obtaining an approximate minimum θ^* in as few iterations as possible. Thus, the value we want to minimize is $f(\theta_T)$ only. However, in practice, w_t needs to be positive for all t in order to guarantee that some optimization is carried out at every iteration for the training to be stable.

RNNs and related architectures are well-suited for L2O: h_t can be interpreted as the hidden recurrent state(s) of the network, and ϕ are the weights and biases of the weight matrices and bias vectors in the network. One of the first applications of deep learning for L2O was the use of a coordinate-wise L2O optimizer,¹² where the same network is applied to each parameter θ_i individually. While this does not couple the parameters and thereby makes the optimizer unable to explicitly use information about the other parameters (i.e. like second-order methods do), it makes it possible to encode the update rule using relatively small neural networks, and thereby to optimize functions with a large or a varying number of parameters.

Training a neural network for L2O is a type of *reinforcement learning*: Instead of starting with a data set beforehand, the model produces candidate solutions and thereby its own data during the training. The task is then to train a model that suggests good candidate solutions. The general work flow reads, for each epoch

1. Define a batch size N_b and sample a set of functions $f^{(i)} \in \mathcal{F}$, $i = 1 \dots, N_b$, from your training family of functions. For each function $f^{(i)}$, define an initial guess $\theta_0^{(i)}$.
2. Calculate $f^{(i)}(\theta_0)$ and $\nabla f_{\theta^{(i)}}^{(i)}(\theta_0^{(i)})$.
3. Set $t = 1$, $L_f = 0$, and initialize $h_0 = 0$. While $t \leq T$:
 - (a) Use eq. II.27 to obtain $\theta_t^{(i)}$ and $h_t^{(i)}$ from $\theta_{t-1}^{(i)}$, $h_{t-1}^{(i)}$, and $\nabla f_{\theta^{(i)}}^{(i)}(\theta_{t-1}^{(i)})$. This update depends on the parameters ϕ .
 - (b) Calculate $f^{(i)}(\theta_t)$ and $\nabla f_{\theta^{(i)}}^{(i)}(\theta_t^{(i)})$.
 - (c) Set $L_f \leftarrow L_f + \frac{1}{N_b} \sum_{i=1}^{N_b} w_t f^{(i)}(\theta_t^{(i)})$.
 - (d) Increase t by one.
4. With the obtained L_f , use back-propagation in time to calculate $\nabla_\phi L_f(\phi)$.
5. Use an optimizer of your choice to update ϕ .
6. Repeat from step 1 until some stopping criterion has been fulfilled (e.g. L_f , on average, stops decreasing; or the maximum number of epochs has been reached).

Observe that the hidden states $h_t^{(i)}$ at each time step may differ for different i - however, the update rule, i.e. the algorithm encoded through m_1 and m_2 (or ϕ , more generally), is the same. Observe also that each $f^{(i)}$ does, in principle, not need to have the same number of parameters. This allows to train and apply a learned optimizer to functions with arbitrary numbers of variables. To monitor the performance, we can observe how well the learned optimizer performs on functions $f \in \mathcal{F}$ that have never been part of the training, but that belong to the same family of functions the model trains on. In addition, we can test the extrapolative performance, i.e. we can test how well the learned optimizer performs on functions $g \in \mathcal{G}$. In principle, one can define a hierarchy of families of functions $\mathcal{F} \subset \mathcal{G}_0 \subset \mathcal{G}_1 \dots$. For the Rothe method, for example, $\mathcal{F}$ can be the Rothe error function in a range of time points $[t_{start}, t_{end}]$ for one field strength, $\mathcal{G}_0$ can be the Rothe error function for all time steps, $\mathcal{G}_1$ can be the Rothe error function for all time steps and all field strengths, and $\mathcal{G}_2$ could be the Rothe error function for several molecular systems at different field strengths. One would naively expect a decreasing performance for each of these cases, as the method is applied to functions that are less similar to the training data.

F. Neural State Space Models (n-SSM)

A state-space model (SSM) describes the dynamics of a continuously evolving process by utilizing an appropriate state equation and a correspondingly appropriate output mapping. Let s_t denote a time dependent state vector such that a state transition in said system can be described as

$$s_t = A s_{t-1} + B u_t, \quad (\text{II.28})$$

where the parametric matrix A controls the state transition and B is a parametric matrix controlling the relationship between an input vector u_t and the current state. At any time t we can map an output vector y_t based on the current system state by

$$y_t = C s_t \quad (\text{II.29})$$

Here, C is a parameterized output matrix. Should the parametric forms of both the state transition matrix and the control matrix have known analytical forms, this method is nothing more than an easily interpretable white-box model which often collapses into a set of ODEs.³⁰ However, for complex dynamics, this is typically not the case and we may take advantage of the fact that the matrices A, B and C all can be learned. This data-driven, black-box type modeling allows us to extract valuable information about the unknown system dynamics. We refer to this as a *neural* state space model (n-SSM).

In this study, we propose that we may recast the optimization problem described by Equation II.27 as a controlled dynamical process wherein the state matrix and control matrix can be trained to update gradients $\nabla_{\theta} f(\theta)$ in such a way that it minimizes a pre-defined loss function. Observe that by replacing u_t in Equation II.29 with gradients $\nabla_{\theta} f(\theta)$, and the output y_t with spatial updates $\Delta\theta$, we have by Equation II.29 recovered a learning policy for optimization. Akin to the LSTM model described previously, the state vector s_t contains the optimization context (gradients, steps, etc.), but is now dynamically controlled by the transition matrix A .

1. Recurrent n-SSM

At the level of complexity described up until now, n-SSMs on recurrent form are very similar to RNNs. The state vector acts like a memory, retaining the context of the dynamical process, and allowing for iterative updates of the dependent variable(s). This simultaneously means that we may construe n-SSMs as linear processes in their simplest form. Unlike RNNs, there is no explicit non-linearity introduced in the iterative process. By this the complexity of the computational flow is reduced, yet so is the ability to generalize the model. What is common, however, is to construct the transition matrices as learnable MLPs which implicitly takes advantage of non-linear transitions during the learning process. By setting

$$A \equiv f_A(\phi_A) \quad (\text{II.30})$$

$$B \equiv f_B(\phi_B) \quad (\text{II.31})$$

$$C \equiv f_C(\phi_C) \quad (\text{II.32})$$

where $f(\phi)$ is some parametric representation of matrices A, B and C , such as a neural network, we effectively retain the simple linear structure of Equation II.29 while at the same time allowing for very complex intermediate dynamics within the parametric matrices. Specifically in this project, we define the matrices A, B and C as deep neural networks (DNNs) with non-linear activation functions to add complexity to the update dynamics. In the same respect, the use of parametric variables ϕ is intentional as they now, like before, may be interpreted as the learnable variables fed into Equation II.27. The settings provided in Equations II.30-II.32 are general and non-definite. The input mapping B may very well be stationary, i.e. not a DNN, such that $B \equiv B\sigma(u_t)$ for some non-linear function σ , as is the case for all matrices in this formalism. The business of defining the transition matrices is context specific. The recurrent form n-SSM may be summarized by

$$s_t = f_A(\phi_A)s_{t-1} + f_B(\phi_B)u_t \quad (\text{II.33})$$

$$y_t = f_C(\phi_C)s_t \quad (\text{II.34})$$

2. Convolutional n-SSM

The recurrent form n-SSM allows, in certain contexts, for a convolutional representation which dramatically lowers the computational intensity of modeling a system using n-SSMs. If we assume that the transition matrices A, B and C are time-invariant for a given context window L , i.e. fixed for $t = 0, 1, \dots, L$, we should observe that

$$s_t = A^{t-1}Bu_0 + A^{t-2}Bu_1 + \dots + ABu_{t-2} + Bu_{t-1} \quad (\text{II.35})$$

$$s_t = \sum_{k=0}^{t-1} A^{t-1-k}Bu_k \quad (\text{II.36})$$

for $k = 0, 1, \dots, t-1$, and since $y_t = Cs_t$

$$y_t = \sum_{k=0}^{t-1} CA^{t-1-k}Bu_k \quad (\text{II.37})$$

by which we construe y_t as a convolution of the input sequence $u_0, u_1, \dots, u_{t-1}$. By re-indexing the sum $j = t-1-k$, this is equivalent to

$$y_t = \sum_{k=0}^{t-1} CA^kBu_{t-1-k} = \sum_{k=0}^{t-1} K_k u_{t-1-k} \quad (\text{II.38})$$

The kernel K_k is learnable through the optimization of A, B , and C during training, typically via backpropagation.²⁰ For a sequence length L , the kernel can be precomputed for $k = 0, 1, \dots, L-1$, enabling efficient computation using Fast Fourier Transforms (FFTs) with complexity $\mathcal{O}((\log L)S(B+S))$, compared to the $\mathcal{O}(LBS^2)$ per time step of the recurrent form, where B is the batch dimensionality and S is the state vector dimensionality.²¹

3. Sparse Representation of State Matrices

Structured state space models, such as the S4 model, enhance the efficiency of the convolutional form by imposing specific structures on the matrices A, B , and C . These structures, such as diagonal or low-rank parameterizations, reduce the computational cost of generating the kernel $K_k = CA^k B$ and ensure stable dynamics.²⁰ Consider a diagonal $A = \text{diag}(a_1, a_2, \dots, a_n)$, where $a_i \in \mathbb{C}$ are learnable parameters. The matrix power may be taken as

$$A^k = \text{diag}(a_1^k, a_2^k, \dots, a_n^k), \quad (\text{II.39})$$

making the kernel computation

$$K_k = CA^k B = \sum_{i=1}^n c_i a_i^k b_i, \quad (\text{II.40})$$

where $B = [b_1, \dots, b_n]^T$, $C = [c_1, \dots, c_n]$, and c_i, b_i are learnable²¹. This diagonal structure reduces the complexity of computing A^k from $O(n^3)$ for a general matrix to $O(n)$, enabling fast precomputation of K_k for a sequence length L .

Stability is ensured by constraining the eigenvalues a_i such that $|a_i| < 1$, preventing exponential growth in A^k and allowing the model to capture long-range dependencies.²⁰ Alternatively, low-rank parameterizations, such as $A = A_0 + UV^T$, where A_0 is diagonal and $U, V \in \mathbb{R}^{n \times r}$ with $r \ll n$, further enhance flexibility while maintaining efficiency.²¹ These structures are optimized via backpropagation, updating the parameters (e.g., a_i, b_i, c_i) to learn an adaptive kernel K_k . The resulting convolutional form is highly efficient, with FFT-based computation enabling low computational complexity for sequences, making structured SSMs ideal for tasks like time series forecasting and language modeling.^{21,31}

In this project, a simplified version of the S4 structure is implemented using an $n \times n$ diagonal matrix A , and a low-rank projection of the input mapping $B = B_0 V^T$ where $B_0 \in \mathbb{R}^{n \times r}$ and $V \in \mathbb{R}^{r \times m}$. Here, m is equivalent to the state vector dimension and the rank r is chosen to be $r = 2$. In light of the context of this project (meta-learning, learning-to-learn) neither the recurrent form n-SSM nor the S4 n-SSM are implemented using the convoluted form as there does not exist a predefined input sequence to convolute. It is nevertheless necessary to inspire curiosity for these forms as kernel precomputation in other contexts is a key aspect of model training in highly complex, data-driven environments. Nearly all of the information provided here is tied together in Mamba²² which allows for fast kernel computation using time-variant matrices A, B and C where the matrices also may be dependent on both input and state vectors u_t and s_t .

The overall benefit, and the driving inspiration for employing these models in this project, stems from the flexibility of these models. As presented above, there are many ways to design the state matrices and therein many ways to adapt the models to specific contexts. The authors imagine that it should be possible to include RNN-like mechanisms in these models (Transformer-layer, memory-like functionalities, etc.) without loss of generality. Many recent projects address specifically the issue of lacking memory mechanisms^{21,22} in baseline n-SSM models.

G. Graph Convolutional Networks (GCNs)

1. Graphs, adjacency matrices, graph Laplacians

Let $x \in \mathbb{R}^N$ be a set of structured data. The underlying structure can often be represented as a graph. Each variable $x_1, \dots, x_N$ is represented as a *node*. If there is an inherent connection between variables x_i and x_j , then

there is a *edge* $E(i, j)$ between nodes i and j . As an example, a social network can be represented as a graph, where each node represents a person, and an edge between person i and person j indicates that these people know each other. Graphs are very general structures (i.e. one can assign a strength parameter e_{ij} with each edge $E(i, j)$), but we will restrict our discussion to the case that each node is associated with just one value and edges are binary (i.e. there is an edge or there is not an edge). One can define the $N \times N$ *adjacency matrix* A , where

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge between nodes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases} \quad (\text{II.41})$$

In addition, we define the diagonal *degree matrix* D

$$D_{ii} = \sum_j A_{ij}, \quad (\text{II.42})$$

i.e. D_{ii} is the number of edges of node i . We define the *graph Laplacian* as

$$L = D - A. \quad (\text{II.43})$$

Because L is diagonally dominant, it is positive-semidefinite. It is also meaningful to define the *normalized graph Laplacian*,

$$\hat{L} = I_n - D^{-1/2} A D^{-1/2}, \quad (\text{II.44})$$

where I_n is the identity matrix. $\hat{L}$ can be shown to have eigenvalues in the range $[0, 2]$. Finally, we define the distance $\text{dist}(i, j)$ between nodes i and j as the minimal number of edges one has to cross to get from node i to node j .

2. Graph Convolutional Networks (GCNs)

Using the graph Laplacian, we can define a polynomial *convolution* of the data x of degree M as²³

$$p(\hat{L}; w)x = \sum_{i=0}^M w_i \hat{L}^i x \quad (\text{II.45})$$

where w are weights. It is straightforward to show that $\hat{L}_{kl}^i = 0$ if $\text{dist}(k, l) > i$. This means that a polynomial convolution of degree M will, in general, convolute nodes that are at most M away from one another.

We follow here the description in Ref. 32 to pick one particular type of convolution. A simple first order convolution based on Chebyshev polynomials reads

$$p(\hat{L}, w) = w_0 x + w_1 (\hat{L} - I_N) x. \quad (\text{II.46})$$

To make this depend on just one free parameter, one can set $w = w_0 = -w_1$ and simplify this to

$$p(\hat{L}, w) = w(I_n + D^{-1/2} A D^{-1/2}) x. \quad (\text{II.47})$$

However the matrix

$$I_n + D^{-1/2}AD^{-1/2} \quad (\text{II.48})$$

has eigenvalues in the range $[0, 2]$, thus repeated convolutions can lead to exploding/vanishing gradients. We can define the self-interaction adjacency matrix and the self-interaction degree matrix

$$\tilde{A} = A + I_N \quad (\text{II.49})$$

$$\tilde{D} = D + I_N \quad (\text{II.50})$$

Finally, the normalized convolution operation now reads

$$\tilde{x} = \tilde{C}x \equiv \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}x.$$

Setting $z^{(0)} = x$, one layer in the GCN now reads $z^{(l+1)} = a^{(l)} \left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}z^{(l)}W_c^{(l)} \right)$ where $a^{(l)}$ is an activation function and $W_c^{(l)}$ is a learnable weight matrix. $W_c^{(0)}$ has dimension $1 \times F$, while $W_c^{(l)}$ has dimension $F \times F$ for $l \geq 1$ for a user-defined value F . For $l = L_c - 1$, where L_c is the number of layers, $W_c^{(L_c-1)}$ will have dimension $F \times 1$ to make sure that $z^{(L_c)}$ has dimension $N \times 1$, i.e. that it is a row vector.

3. The T-GCN architecture

Let x be a multivariate time series with a graph structure, with x_t corresponding to the data at time step t . The T-GCN architecture combines recurrent neural networks with graph neural networks.²⁴ The idea is very simple: Instead of putting the "raw" data x_t into an LSTM, we input the convoluted data, i.e.

$$h_t^0 = z_t^{(L_c)}.$$

The index t for $z_t^{(L_c)}$ means that x_t was convoluted using the GCN. In the original T-GCN paper, a gated recurrent unit neural network (GRU) was used, however, due to the relative small size of our data, we will use an LSTM. The idea of the T-GCN architecture is hence that the GCN takes care of the structure between the variables, while the LSTM takes care of the temporal aspect. The LSTM parameters and the GCN parameters are learned jointly.

III. METHODS

A. Test problem: Quadratic functions

As a first test, we consider quadratic functions of dimensionality n ,

$$f(\theta, A, \mu) = (\theta - \mu)^T A (\theta - \mu) \quad (\text{III.1})$$

where A is a positive semidefinite real matrix of dimension $n \times n$ and μ a real vector of length n . The global minimum occurs at $\theta^* = \mu$ (if A is positive definite, the minimum is unique).

To train our L2O model, we follow the setup described in Section II E. At each epoch, we generate functions $f^{(i)}(\theta)$ as follows:

- Randomly select a dimensionality n from the set of dimensionalities for that epoch. Let index i run from 1 to N_b .
- Generate N_b matrices $B^{(i)}$ of dimension $n \times n$, with elements independently sampled as $B_{kl}^{(i)} \sim \mathcal{N}(0, 1)$. Then, set $A^{(i)} = (B^{(i)})^T B^{(i)}$ to ensure positive semi-definiteness.
- Generate N_b vectors $\mu^{(i)}$ of length n , with elements independently sampled as $\mu_k^{(i)} \sim \mathcal{N}(0, 10)$.
- Define the quadratic function as

$$f^{(i)}(\theta) = \left(\theta - \mu^{(i)} \right)^T A^{(i)} \left(\theta - \mu^{(i)} \right). \quad (\text{III.2})$$

- Set the initial guess to $\theta_0^{(i)} = 0$.

Choosing a single dimensionality n per epoch facilitates computational efficiency. For training, we used dimensionalities from $n = 1$ to $n = 20$. The initial guess is somewhat close to the true minimum.

For the LSTM-based L2O optimizer, we built a model that treats dimensions independently of one another, in line with the approach presented in Ref. 12. In order to guarantee scale invariance across dimensions and initial guesses, the value we end up optimizing is

$$\mathcal{L}(\phi) = \mathbb{E}_{f \in \mathcal{F}} \left[\sum_{t=1}^T w_t f(\theta_t) / f(\theta_0) \right], \quad (\text{III.3})$$

i.e. we aim to reduce the relative error with respect to $f(\theta_0)$. Let $\theta_t^{(j)}$ denote the j -th component of the parameter vector at iteration t . Thus, for optimizing an n -dimensional function, we use n LSTMs with identical (shared) weights running in parallel. Consequently, the network does not explicitly capture second-order coupling between parameters. The general design of the network is as follows

- The input $x_t^{(j)}$ to the network is a two-dimensional vector

$$x_t^{(j)} = \left[\log \left(\left| \frac{\partial}{\partial \theta^{(j)}} f(\theta_t) \right| + \varepsilon \right), \text{sgn} \left(\frac{\partial}{\partial \theta^{(j)}} f(\theta_t) \right) \right] \quad (\text{III.4})$$

where $\varepsilon = 10^{-16}$ ensures that the logarithm can be taken.

- The input vector is first processed through a linear transformation of dimension $2 \times n_L$, followed by a nonlinear ReLU activation function, and subsequently through another linear transformation of dimension $n_L \times n_L$.

- We apply n_{hl} hidden LSTM layers, where each hidden layer has an output of dimensionality n_h .
- The final output is the scalar update $\delta_{t+1}^{(j)}$, which is a linear transform of the output of the last layer. The parameter at iteration $t + 1$ is then computed as $\theta_{t+1}^{(j)} = \theta_t^{(j)} + \delta_{t+1}^{(j)}$.

We use the ADAM optimizer with PyTorch’s default parameters. The hyperparameters optimized during training are n_L , n_{hl} , n_h , and the learning rate λ . Although it would be possible to explore additional linear layers, different activation functions, alternative optimizers, or further tuning of the ADAM parameters, the combinatorial explosion of tunable hyperparameters would make a systematic optimization infeasible in practice. Furthermore, there is the unrolling depth T to consider; we discuss its effect on training in the results section.

To find the best model, we used Ray Tune³³ to pick the best parameters. We set $\lambda \in [10^{-4}, 10^{-2}]$, $n_L \in \{2, 8, 32, 128\}$, $n_{hl} \in \{1, 2, 3\}$, $n_h \in \{32, 64, 128, 256, 512\}$, $N_b \in \{8, 16, 32, 64\}$. For the n-SSM models we consider the same parameter space, however we must replace n_h with n_S where n_S is the state size. This parameter is taken as the same set as n_h . The state matrices do not have layers, so for these training modules we do not consider n_l . Additionally, for the S4-SSM models we introduce Rank $\in \{1, 2, 4\}$ which is used to determine the size of the projected input mapping $B_0 V^T$. To train our model, we pick that set of parameters that gives the lowest average train error over the last 20 iterations (to take into account variability in n) at epoch 200 (we train for at most 200 epochs). We consider 128 parameter combinations, making use of Ray Tune’s `ASHAScheduler(reduction_factor=2)`, the learning rate λ is drawn from a log-uniform distribution. *Because we struggled to get reproducible data with Ray Tune, we did not use it for the Rothe data.*

As the system is provided with new data at each epoch, we do not have to consider a test error (as the training error before backpropagation is an unbiased estimate of the generalization error), though we will discuss the performance for $t > T$ and $n > 20$ in the results to see how well the models generalize. For w_t , we consider $w_t = 1/T$ (i.e. constant w_T) and $w_t = k^t / \sum_{k=1}^T k^t$, i.e. the error at deeper iterations becomes exponentially more important.

B. Application of L2O to Rothe’s method for the Hydrogen atom

For Rothe’s method for the Hydrogen atom, we followed a very similar approach as for the quadratic functions, and we will reuse the same type of network structure and loss function. However, we made use of the fact that the 4 parameters that represent each Gaussian are inherently and strongly coupled. In particular, the dimensionality of the problem is $4M(t)$, where $M(t)$ is the

(time-dependent) number of Gaussians. We run $M(t)$ networks in parallel, and each network takes as input the logarithm of the absolute value of the derivative, and the sign of derivative, for all 4 parameters (i.e. the input is 8-dimensional). The output δ will then be 4-dimensional. In addition, for numerical reasons, we multiply the output δ by a factor e^{-10} , as the Rothe updates are generally very small – without this factor, the Rothe error blows up and optimizing the network becomes very hard. Other than that, the layout of the LSTM network for Rothe’s method is identical to the one we used for quadratic functions. The goal, is to find the parameters $\theta_{i+1}^{\text{opt}}$ that minimize the Rothe error $r_{i+1}(\theta)$ at time t_{i+1} .

To train our L2O optimizer, we have, in principle, 9 data sets available, for the field strengths $E_0 = 0.03$, $E_0 = 0.06$, and $E_0 = 0.012$. For each field strength, we have a low-quality data ($q = 1$), mid-quality data ($q = 2$) and high-quality data ($q = 3$), where a higher quality implies that more Gaussians were used. In each data set, we simulated from $t = 0$ a.u. to $t = 330$ a.u., with a time step of $h = 0.2$ a.u. (a.u. stands for atomic units). Computationally, stronger fields and higher qualities are more demanding, as are larger values of t , both in terms of the computation time it takes for a single iteration (i.e. an evaluation of the Rothe error and gradient) and the number of iterations necessary.

As we want to train the network on realistic problems, we use as an initial guess for the Gaussian parameters θ_{i+1} at t_{i+1} the solution from the previous time step, i.e. θ_i^{opt} . In order to make batched gradient descent possible, we have to make sure that each element in the batch contains the same number of Gaussians. To do this, we sample t_i from a time range $t_i \in [t_{\min}^{\text{sample}}, t_{\max}^{\text{sample}}]$, and consider the N_b time steps $t_i, t_i + h, \dots, t_i + (N_b - 1) \cdot h$ when using gradient descent if the number of Gaussians is the same for all time steps. If not, we suggest a new t_i and repeat the procedure. This procedure is relatively simple, and while it may exclude a few data points from the training set, the number of Gaussians changing more than once over 4 – 8 iterations does not happen often. However, this procedure makes it possible that some time steps enter the training set several times, hence we have added L^2 regularization to the weight parameters of the network, with regularization strength λ_{L^2} .

As calculations using Rothe’s method are computationally costly and because training hundreds of models and compare their performance is prohibitively expensive, we decided to keep several parameters fixed. In particular, we set $T = 10$, $k = 1$, and the number of hidden layers equal to 3. Also, we set $n_{hl} = n_L = 256$. This gives us 3 parameters to vary: λ_{L^2} , N_b as well as the learning rate λ . We use $\lambda \in [10^{-4}, 10^{-2}]$, $\lambda_{L^2} \in [10^{-6}, 10^{-2}]$, and $N_b \in [1, 2]$.

We consider the following two models:

1. A narrow model, where we train on data for $E_0 = 0.06$, $q = 2$ for $t \in [180, 200]$ and use as test how well it generalizes for $t \in [200, 210]$. We consider two types of validation sets, one where

$t \in [210, 250]$, and one where we instead change q , i.e. $t \in [180, 200]$, $q = 3$. We train for 500 epochs.

2. A broad model, where we train on data for $E_0 \in \{0.03, 0.06, 0.12\}$ (low quality) for $t_i \in [180, 260]$ and use as test how well it generalizes for $t \in [260, 280]$ for all values of E_0 considered. We consider two types of validation sets, one where $t \in [280, 330]$, and one where we instead change q , i.e. $t \in [180, 260]$, $q = 2$. We train for 1000 epochs, at each epoch randomly picking a value of E_0 .

The validation sets were chosen as two ways to use L2O for Rothe's method - either using L2O for out-of training time points, or for higher qualities. As evaluating the test error is computationally expensive, we only evaluated it at every 20th epoch during training. We ended up picking the hyperparameters that gave the lowest test error at any possible epoch. As we are only interested in the error at iteration T , this is what we mean by the test error. For the test, we use 20 evenly spaced time points t_i in the test range (always the same). We will then train the model with the best hyper-parameters for 1000 epochs to see if it can be improved further, and pick the best epoch for the results we present.

C. Prediction of an initial guess using with GCNs

As a second application of machine learning to Rothe's method, we consider using GCN's to predict the nonlinear parameters at the next time step (the linear coefficients can be obtained cheaply when the nonlinear parameters are known). We do this for the Henon-Heiles potential, as the number of Gaussians are constant, making the datasets uniform in size. We consider the following dataset:

- We have $D = 3$, $M = 28$, with $t_0 = 4.97$, $t_f = 99.99$ and $h = 0.01$. With 18 parameters per Gaussian, this corresponds to 9503 time points and 504 nonlinear parameters per time point. Let $\theta(t_i)$ be the nonlinear parameters at time point $t = t_i$. In addition, let $\Delta_i = \theta(t_{i+1}) - \theta(t_i)$ be the update of the nonlinear parameters.

To set up the adjacency matrix A , we consider the following graph structure: Let p_{ij} be the i 'th nonlinear parameter of Gaussian j . There will be an edge between p_{ij} and p_{kl} precisely if $i = k$. This means that all nonlinear parameters are coupled to the other nonlinear parameters corresponding to the same Gaussian. We also considered adding an edge whenever $j = l$. However, with this, a single application of the convoluting matrix $\tilde{C}$ would lead to all nodes being correlated, something that could wash out the information due to repeated convolutions through repeated smoothing.

Our application of T-GCN is to predict the update $\tilde{\Delta}_i$ at the next time step, given the T previous updates.

Hence, we are interested in next-step prediction. That is, we want to learn a function

$$f_R(\Delta_i, \Delta_{i+1}, \dots, \Delta_{i+T-1}) = \tilde{\Delta}_{i+T} \approx \Delta_{i+T}. \quad (\text{III.5})$$

We encode the function f_R as a T-GCN. However, we found that building a model that predicts each value of the 504-dimensional vector Δ_{i+T} is extremely prone to overfitting. Hence, we decided to simplify the output function in such a way that $\tilde{\Delta}_{i+T}$ is a linear combination of the previous updates, that is, as an autoregressive (AR) model

$$\tilde{\Delta}_{i+T} = \sum_{j=i}^{i+T-1} \alpha_j(\Delta_i, \dots, \Delta_{i+T-1}) \Delta_j \quad (\text{III.6})$$

where α_j depend on the input via the T-GCN model. It should be understood that this is *not* a linear model. Due to the underlying neural network, α_j are not constants, but they are themselves functions of the input, as is indicated in eq. (III.6). As we treat Δ_i as a time series, we obtain, by the last equation of eqs. (II.18)

$$\alpha = W_y h_t^L + b_y$$

where h_t^L is the final hidden state of the last layer of the LSTM of size n_h , and W_y is a (trainable) matrix of dimension $T \times n_h$. Furthermore, due to the underlying GCN, the input to the LSTM are the T time steps $z_i^{(L_c)}$, i.e. the convolution of the Δ_i . The underlying assumption of the AR model is that we represent Δ_{i+T} as a linear combination of the previous updates — if now the real Δ_{i+T} is almost orthogonal to $\Delta_i, \dots, \Delta_{i+T-1}$, then the architecture won't be able to predict a $\tilde{\Delta}_{i+T}$ that is similar to the real Δ_{i+T} .

To train the t-GCN, we do a sweep over the following parameters: the unrolling depth $T \in \{5, 10\}$, the number of hidden layers $n_{hl} \in \{1, 2, 3\}$, the number of graph convolutions $L_c \in \{2, 3\}$ the batch size $N_B \in \{16, 32, 64\}$, the number of nodes in the hidden layers $n_h \in \{64, 128, 256\}$, and the L2 regularization strength for the weights $\lambda_{L^2} \in \{10^{-6}, 10^{-5}, 10^{-4}\}$. To not get an explosion in the number of parameters, we set the learning rate $\lambda = 10^{-3}$ and the number of "nodes" in the GCN $F = n_h$. As activation functions for the GCN, we use the ReLU function. Here, we do not make the initial hidden states trainable. While PyTorch has a t-GCN implementation, we chose to write our own (in PyTorch) for increased flexibility - though we use the vanilla LSTM implementation in PyTorch, hence, the initial hidden and cell state are not trainable.

Furthermore, we will do the same parameter sweep for a pure LSTM (barring the GCN-related parameters) to see whether the t-GCN gives improved results. That is, instead of $z_i^{(L_c)}$ entering into the LSTM, we just use the raw data Δ_i directly.

As training set, we use the first 1000 time points in the set. As a test set, we use the average performance on the

next 200 time points. We will visualize the performance on the whole data set in the results. As a baseline, we use $\tilde{\Delta}_{i+1} = \Delta_i$, i.e. assume that the parameters will change exactly the same way as they did before. If our model cannot outperform the baseline, then it is not applicable for Rothe’s method.

We scaled our data in such a way that the training data has mean 0 and variance $\sigma^2 = 1$ for each of the 504 variables.

IV. COMPUTATIONAL DETAILS

All training of Neural Networks was done using Pytorch.²⁹ We exclusively used ADAM with PyTorch’s default parameters (except for the learning rate) as an optimization algorithm to carry out the training of our models. We used `scipy.optimize.minimize` for the BFGS algorithm,³⁴ which serves as a reference, with default parameters except for a custom initial Hessian H_0 ,³⁵ which we set to

$$H_0^{-1} = \text{diag} \left(\frac{1}{|\nabla f(\theta_0)| + 10^{-14}} \right) \quad (\text{IV.1})$$

for all simulations considered, where θ_0 is the initial guess for the parameters.

V. RESULTS

A. LSTM for quadratic functions

In figure 1, we compare three learned L2O optimizers, obtained using three different unrolling depths $T = 5, 10, 20$ to two classical optimizers: ADAM using four different learning rates, and an approximate quasi-Newton method, BFGS. We trained the three L2O models independently, i.e. they might have different architectures, picking the one that Ray Tune returned. To produce that data, we initialized 1000 random problems as described in section III A, using $n \in [1, 20]$ for training and $n \in [21, 50]$ for testing. The lines represent the median relative reduction in the median, with the intervals indicating the 25th and the 75th quartile, respectively. Figure 2 shows the corresponding data using $T = 20$, using $k \in \{1, 1.2, 1.5\}$.

For all T and all k , we get much faster convergence than ADAM with the optimal learning rate λ initially, but in all cases, the decrease in error flattens out after between 10 ($T = 5, k = 1$) and 80 ($T = 20, k = 1$) iterations. We observe that for all T values considered and $k = 1$, that the initial decrease is about equally fast for both training and testing, but the results flatten out faster for $T = 5$ than for $T = 10$, and faster for $T = 10$ than for $T = 20$. We observe an excellent generalization to the test data, with the curves looking essentially the

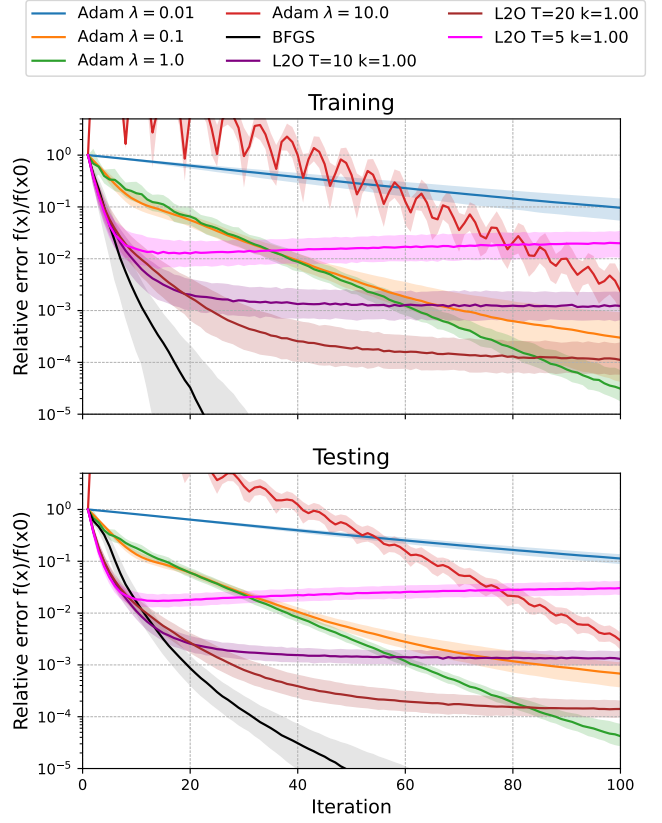


FIG. 1: L2O optimizers applied to the train set using dimensions $n \in [1, 20]$, and the test set $n \in [21, 50]$ compared to standard methods (ADAM using different learning rates λ and BFGS). Here, we used $T \in \{5, 10, 20\}$ and $k = 1$. The data was produced using 1000 random problems. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

same for training and testing. The same is true for the ADAM results, while BFGS is somewhat worse (but still performs best on the long run). For the test data, we see that BFGS is initially outperformed by the L2O optimizers. We observe that larger unrolling depths T lead to improved results, and we also observe that $k = 1$ seems to yield the best results. However, this is not a definite conclusion, as it might simply be the case that we did not manage to find network parameters (both weights, biases and hyperparameters) that would give rise to the best results. In figure 3, we show the training loss for the best performing model as function of the number of epochs (because we struggled with RayTune, we didn’t manage to extract the trained model or the initial parameters before training, so this is from a corresponding model with the same hyperparameters where the initial parameters follow the same distribution). This shows that the learning of these L2O models is not straightforward and can be extremely noisy. However, it also shows that the learning is quite efficient, i.e. that approximately 100 epochs

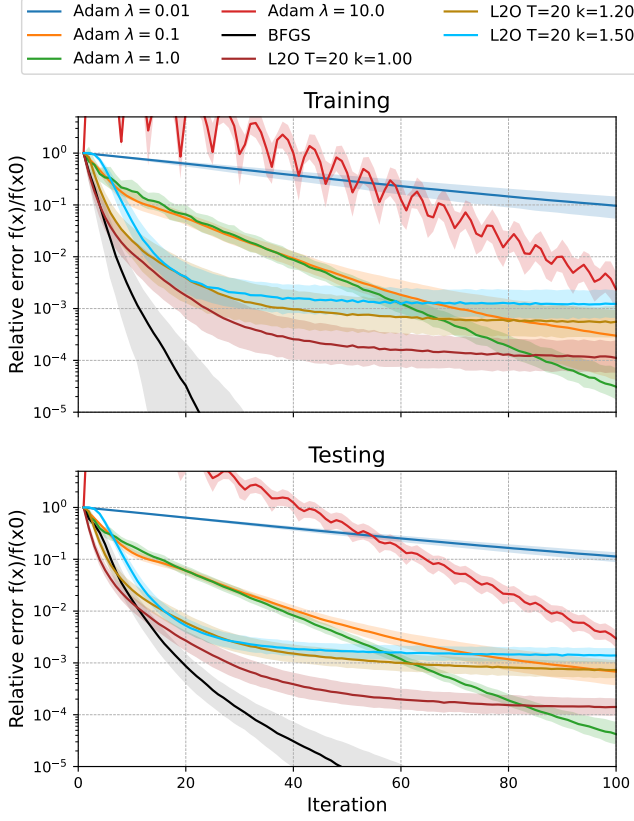


FIG. 2: L2O optimizers applied to the train set using dimensions $n \in [1, 20]$, and the test set $n \in [21, 50]$ compared to standard methods (ADAM using different learning rates λ and BFGS). Here, we used $T = 20$ and $k \in \{1, 1.2, 1.5\}$. The data was produced using 1000 random problems. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

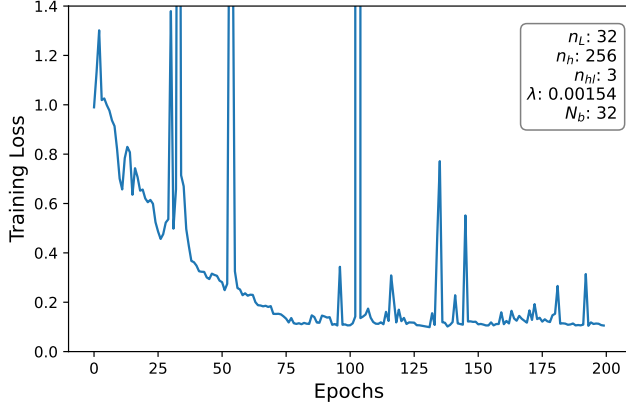


FIG. 3: Training loss as function of the number of iterations for $T = 20$, $k = 1$. The model parameters as described in Section III A are shown in the figure.

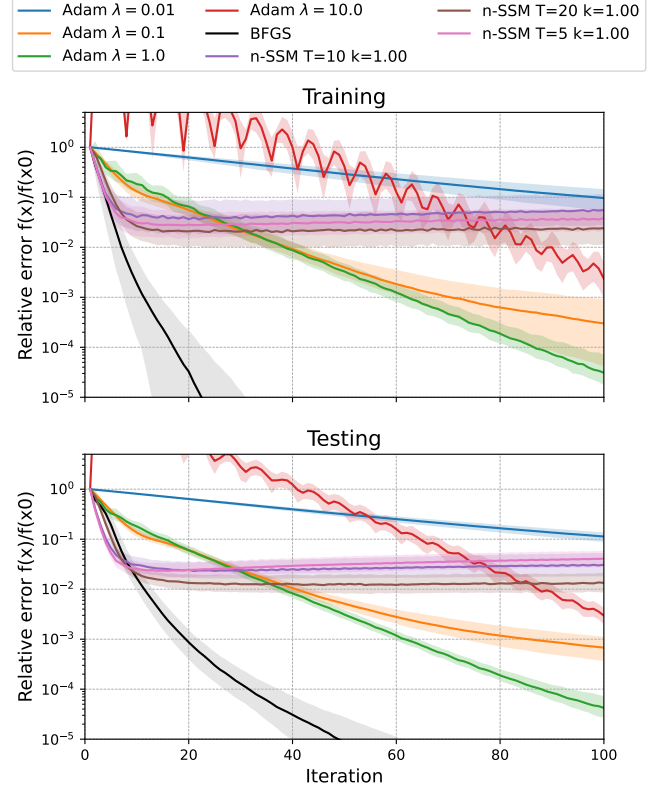


FIG. 4: Recurrent form n-SSM optimizers applied to the training set using dimensions $n \in [1, 20]$, and the test set $n \in [21, 50]$ compared to standard methods (ADAM using different learning rates λ and BFGS). Here, we used $T \in \{5, 10, 20\}$ and $k = 1$. The data was produced using 1000 random problems. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

are sufficient to reach (noisy) convergence. As described in the theory section, overfitting isn't a problem, as no problem has been seen previously.

B. n-SSM for quadratic functions

The training process for all n-SSM models presented in this project is performed in the same way as for the LSTM models, using RayTune to determine the optimal model parameters. The tuned (hyper)parameters are exactly the same as for the LSTM models, with the obvious exception of `hidden_layers` being replaced by `state_size` which does not have `num_layers` dimensions.

TABLE I: Tabular overview of optimal hyperparameters for best-performing n-SSM and S4-SSM models at $T=20$.

Model type	n-SSM	S4-SSM
T	20	20
n_l	8	8
n_s	64	256
n_b	16	16
λ	$1.17 \cdot 10^{-4}$	0.00114
Rank	NA	2

Akin to the LSTM-based optimizers, the recurrent form n-SSM optimizers as presented in Figure 4 exhibit behaviors such as improved convergence rates compared to ADAM, and on-par performance compared to BFGS in the very first iterations. Deeper unrolling depths T improve on the results both in the training and testing scenario. Yet, these models dramatically underperform compared to the LSTM-based optimizers, achieving relative errors several magnitudes of order larger than the LSTM-L2O models. It is also obvious that the recurrent form n-SSM models lack the ability to retain training context over longer sequences as seen by the much-too-fast plateauing each optimizer displays, even before 20 iterative steps have been taken. Albeit, the models seem to have generalized their context in such a fashion that they too perform well on higher-dimensional quadratics. Moreover, we observe a much smaller spread in the overall performance across different unrolling depths. We expect this to be a direct result of the model structure. The recurrent behavior of the model implies a lack of long-range dependence, such that deeper unrolling depths should result in low information retention towards the end of T . We are nevertheless impressed with the overall performance of this baseline-type n-SSM model for one reason; it is much more pedagogical than many ML models. Their straightforward architecture, with clear ties to well known mathematical and physical concepts, make them a perfect introduction to SSM-like architectures. There exist many ways to build more sophisticated and high-performing architectures as discussed in Section II F 3.

In contrast to the recurrent form n-SSM optimizers, the structured matrices in n-SSM models based on S4-like models are specifically designed to avoid the exponentially decaying information for longer sequences. We may to some extent observe this in Figure 5, where performance increases for longer unrolling depths T . However, the problem of plateauing performance persists in these models, as well. The hyperparameters provided in Table I give us some slight indication towards the major differences between the LSTM-based models and the n-SSM models. We immediately observe that at unrolling depth $T = 20$ both n-SSM models perform optimally with smaller parameter dimensions compared to the LSTM-based model, yet they never perform as well as the LSTM-based model. This might seem like a contra-

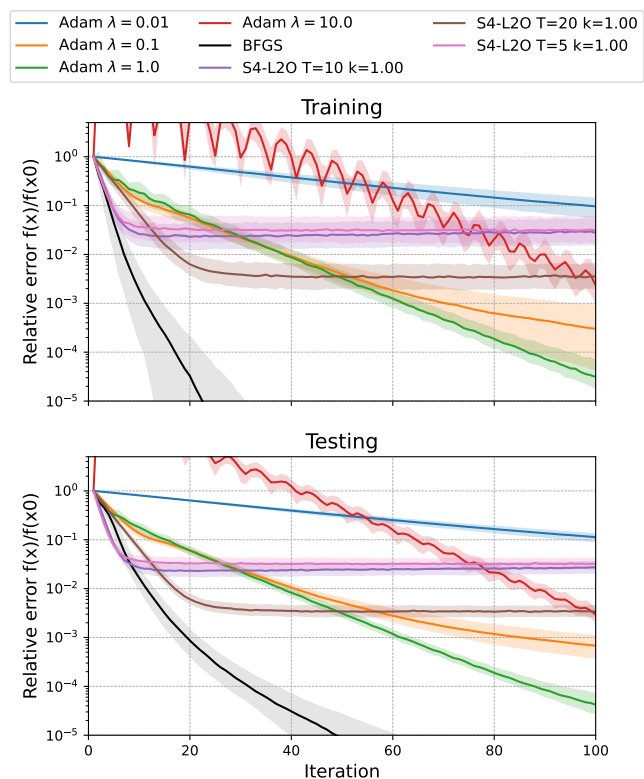


FIG. 5: Structured form (S4) n-SSM optimizers applied to the training set using dimensions $n \in [1, 20]$, and the test set $n \in [21, 50]$ compared to standard methods (ADAM using different learning rates λ and BFGS). Here, we used $T \in \{5, 10, 20\}$ and $k = 1$. The data was produced using 1000 random problems. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

diction, yet is likely a result of the model characteristics. Firstly, the models based on LSTM has a much greater potential for very complex modeling of dynamics, given sufficiently large matrices and finely controlled gradients. The level of complexity is much harder to obtain using n-SSM models in the same parameter space, as the long-range dependencies are not captured in the same way. Simultaneously, the linearity and global context window allows n-SSM models to perform well using much fewer parameters, a key strength of these models. This quickly leads to a cost-yield balance which a user must consider. The most glaring issue still remains the combination low performance and a 'quick-to-forget' model structure. Nevertheless, it was for this project still an interesting and necessary investigation as these models offer not only machine learned models which are easily bridged with mathematics and natural sciences, but also architectures that are highly flexible and modifiable. Additionally, and in contrast to LSTM-based models, n-SSM models can easily be implemented as massively parallel models due to their linear recurrent form. n-SSM models, while well

known in the deep learning community, are still under-represented in meta-learning and specifically learning-to-optimize contexts.

C. L2O for Rothe's method with the Hydrogen atom

For the narrow model, we found that the best parameters where $N_b = 2$, $\lambda = 10^{-3}$, $\lambda_{L^2} = 10^{-5}$. The results for the train, test and the two validation data (compared to BFGS) are shown in figure 6. The data is based on 50 random time points sampled from the respective range, see Section III B.

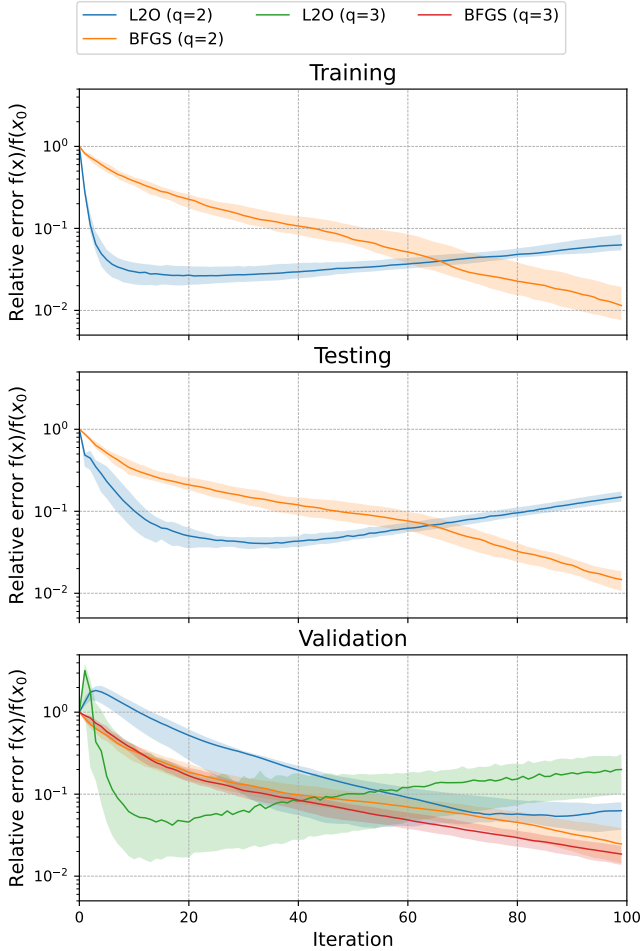


FIG. 6: L2O applied to the Rothe error, trained on $q = 2$, $E_0 = 0.06$ in the time range $t \in [180, 200]$ (narrow case). A description of the different training, testing and validation sets is found in Section III B. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

We observe that the learned optimizer manages to reduce the error for all the sets considered, showing that it works. In three out of four cases (except for the $q = 2$ validation), it outperforms BFGS for few iterations. How-

ever, we also observe that the Rothe errors start going up after a minimum has been reached for all 4 data sets, showing that the optimizer has not learned any correct "stopping" behavior. We observe that the position of the minimum also shifts to the right for both the testing and the validation sets, showing that the optimizer can optimize for longer than just 10 iterations. In the validation data, we see that the initial steps taken at the first iteration(s) are too large, which can be considered as type of misgeneralization, which we assume is due to taking too big initial steps – however, the learned optimizer manages to make up for that misbehavior later (and for the $q = 3$ validation, still outperform BFGS).

We clearly see that the optimizer performs best on the training set, which is not surprising, as it is trained on that. We see that the performance is not as good for the testing set, which resembles the training set a lot, and even worse for the $q = 2$ validation set – showing that the optimizer performs worse the more "different" an optimization problem becomes.

Finally, we observe that for the $q = 3$ validation, that the convergence to the local minimum is very fast compared to BFGS (converging to a relative error of $\sim 2 \cdot 10^{-2}$, on median, almost 5 times faster than BFGS), but at the cost of a very high confidence intervals – the BFGS has a much smaller intervals, showing that it treats problems similarly; while the learned optimizer has very large confidence intervals, showing that it performs much better on some problems than on others.

Figure 7 shows the corresponding data for the broad model. We found parameters $N_b = 1$, $\lambda = 10^{-3}$, $\lambda_{L^2} = 10^{-6}$. The data is based on 50 random time points and random values of $E_0 \in \{0.03, 0.06, 0.12\}$ sampled from the respective range, see Section III B.

Many conclusions are similar to the narrow case, however, there are a few important differences. First of all, the learned optimizer has now learned a stopping behavior, which we observe for all four data sets considered. This indicates that a larger variety of training points from different problem types (i.e. different values of E_0) and a larger variety of time points improves the optimizer's behavior once an approximate minimum has been reached. However, this seems to come at a prize – namely, the increased variance in the error, which is much larger than for BFGS.

We observe that the performance on the test set is similar to the performance on the training set, both outperforming BFGS while the validation on the $q = 1$ validation set (far out-of-sample time points) starts off by taking a far too big step, similar to the narrow case, but manages to resolve it later on. However, the $q = 2$ validation set (higher quality than training) does not show that behavior and clearly outperforms BFGS – again, similar to the narrow case, the optimizer generalizes better to higher qualities than to different time points. It hence seems that the optimization for different qualities is in some sense more similar than the optimization for different time points.

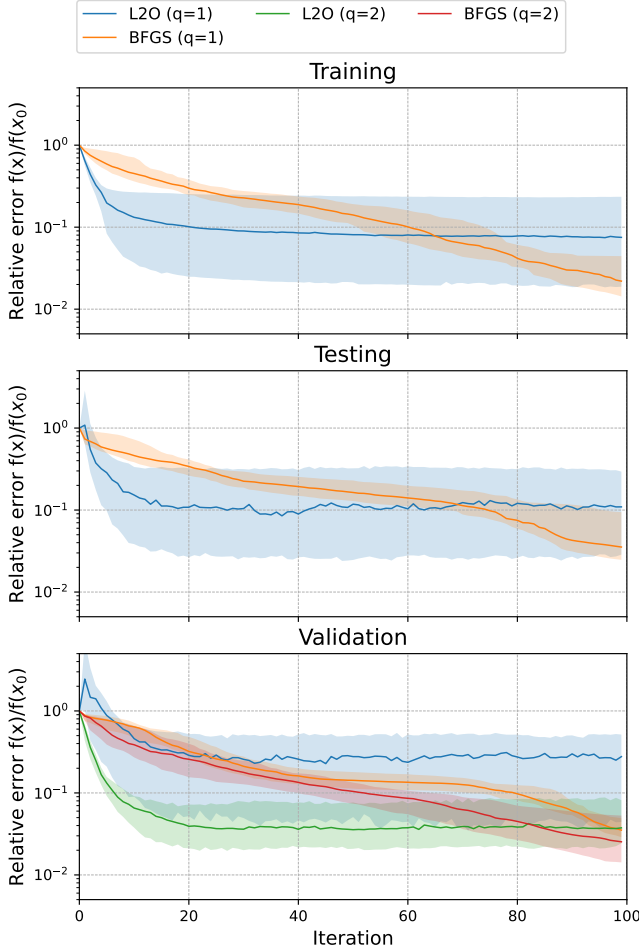


FIG. 7: L2O applied to the Rothe error, trained on $q = 1$ and all possible values of E_0 in the time range $t \in [180, 260]$ (broad case). A description of the different training, testing and validation sets is found in Section III B. The thick lines express the median, while the intervals represent the 25th and the 75th quartile, respectively.

D. T-GCN to predict nonlinear coefficients

In figs. 8 and 9, we show the (normalized) mean square error obtained using an LSTM and the t-GCN for $T = 5$ and $T = 10$. The meta-parameters that define the model are shown in table II. We observe that both $T = 5$ and $T = 10$ outperform the baseline for both the LSTM and the t-GCN, in both training, testing and, largely, validation. There are large oscillations in the baseline approach, which sometimes works very well and sometimes not so well – those oscillations are largely suppressed in our learned approaches, especially in the training data, though they are still present to some degree. For both models, using $T = 5$ is insufficient to suppress those oscillations in the testing data and the validation data – however, using $T = 10$, the oscillations are largely gone,

TABLE II: Meta-parameters for the t-GCN and the LSTMs used that produce figs. 8 and 9, respectively.

Model type	LSTM	LSTM	t-GCN	t-GCN
T	5	10	5	10
N_B	16	32	64	32
n_{hl}	3	3	1	2
n_h	256	128	256	128
L_c	–	–	3	3
λ_{L^2}	0.0001	0.001	0.0001	0.0001
Epoch	1960	1340	1940	1360

showing that $T = 10$ is a good degree for the autoregressive model. This claim is further supported by the observation that $T = 5$ performs equally well for the t-GCN architecture and the LSTM architecture, as does $T = 10$. Indeed, we observe that there is no real advantage here to use the t-GCN – whether this is due to the fact that we have not chosen a very good graph structure defined through the adjacency matrix A , or whether the autoregressive structure does not profit from it, is hard to say. Our data has no inherent graph structure and is not very high-dimensional, which might be a reason for this. Based on the fact that the LSTM is much easier to implement and much faster to train (as no convolutions need to be carried out), we consider the LSTM to be the superior architecture here. We observe regions where all considered methods to suggest new parameters do not work very well – for example, at time step t_i , $i \approx 550$ and at iteration t_j , $j \approx 1550$. Because this behavior is model-independent and also occurs in the training data, we believe that this is due to an inherent shortcoming of the autoregressive ansatz in those particular regions.

VI. CONCLUSION

We have shown that L2O can be applied to high-dimensional optimization problems effectively. For high-dimensional quadratic problems, our LSTM model managed to outperform ADAM, and show performance on par with BFGS at least for the first ~ 20 iterations. For the actual Rothe problem, we have seen that a well-trained LSTM model can outperform BFGS for problems that resemble the training data – in particular, it can generalize to time points that are not too far away from the training data, and it can generalize to runs with a higher number of Gaussians at convergence speeds far outperforming BFGS initially. Nevertheless, our contribution is a proof-of-principle, and there is a lot of work left to be done to be reliably able to apply L2O to Rothe’s method in a research context. Most importantly, in the future, we aim to train an optimizer that “knows when it’s stuck” by implementing a switch from the L2O optimizer to a BFGS optimizer, or implementing an L2O model that has guaranteed convergence properties.¹⁵ A simple line search along the optimization direction suggested by the optimizer could also speed up convergence,

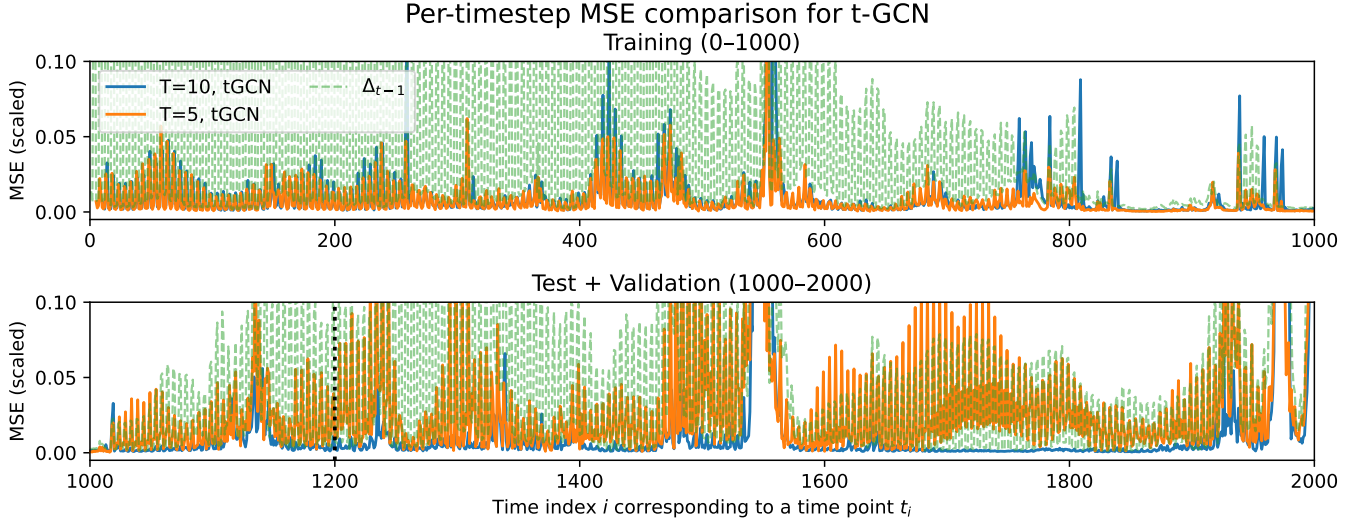


FIG. 8: Scaled mean squared error (MSE) of the starting guess at time point t obtained using the heuristic updated rule Δ_{i-1} , a t-GCN with $T = 5$ and $T = 10$ over 3000 time points. Training was from time points 0 to 1000, testing was carried out on time points 1000 to 2000, which is indicated using the black dotted line.

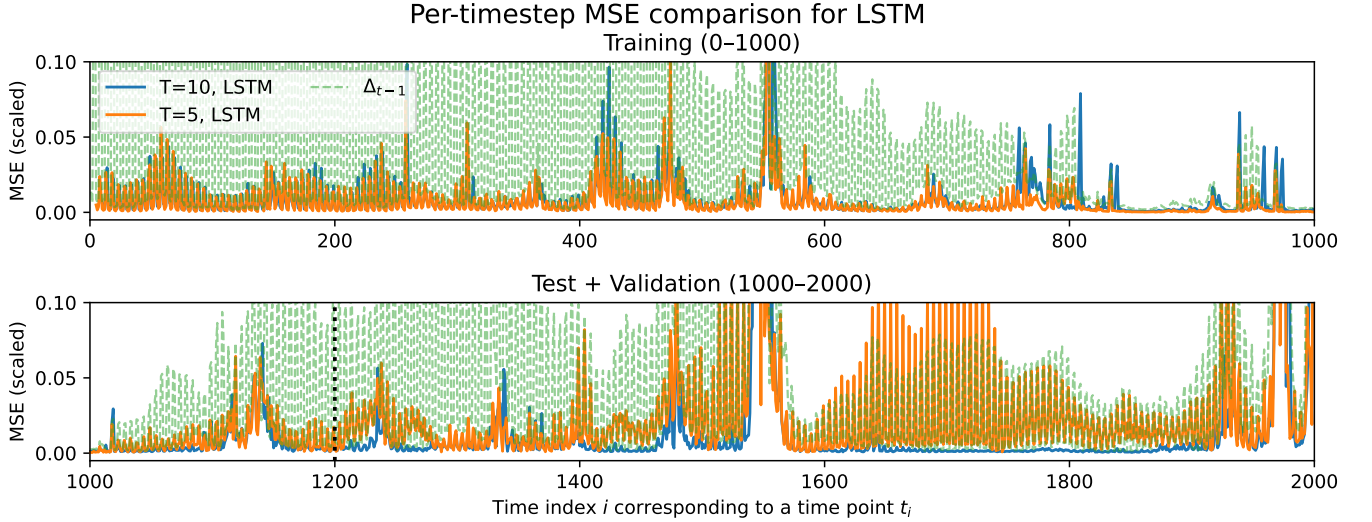


FIG. 9: Scaled mean squared error (MSE) of the starting guess at time point t obtained using the heuristic updated rule Δ_{i-1} and an LSTM with $T = 5$ and $T = 10$ over 3000 time points. Training was from time points 0 to 1000, testing was carried out on time points 1000 to 2000, which is indicated using the black dotted line.

especially at the first iteration, where no gradient information has been accumulated. In addition, further experimenting with different designs of L2O optimizers and loss functions might give rise to even better models that converge even faster, or manage to reach a better minimum. Furthermore, many of our assumptions from the previous section should be further explored across different systems to assess the effect not only on the Hydrogen atom, but Rothe’s method in general.

We have shown that n-SSM models also can be applied in the context of high-dimensional L2O. While both the LSTM-based models and BFGS perform orders of mag-

nitude better, the n-SSM models and specifically the S4-like n-SSM models outperform common optimizers like ADAM within a given information window. In light of this, we mention and reference works which implement additional complexity in these models, intentionally steering clear of these implementations ourselves in order to retain the simplistic form of n-SSM models for this study. We observe, too, that using structured, low-dimensional matrices in an S4-like formalism results in much better generalization for unseen quadratic functions and a much clearer relationship between unrolling depth and performance. Thus, we are confirming that

the lack of context retention in simple n-SSM implementations is likely a result of decaying information strength in each unrolling step during training. The sub par performance of these models compared to the LSTM models is the reason for why these models have not been trained on Rothe data as the LSTM models have, due to limited time and computational resources. It has been shown in other works, however, that these models do possess the ability to learn dynamics using very limited input sequences³⁶ which inspires the potential for further investigation into their role in L2O-based models, making the outlook for such models positive.

We have shown that both LSTMs and t-GCNs can be used to predict excellent starting guesses at the next time step when an autoregressive structure is used, with the L-STM and the t-GCN giving results of the same quality. A main drawback of this approach is that our model was very inflexible in the sense that it did not allow a variable number of Gaussians, and neither a variable number of parameters per Gaussian. In the future, we would like to explore whether size-invariant approaches that embed Gaussians with different number of parameters, such as deep sets or set transformers, can be used in order to train a model that is applicable across different number of Gaussians and across different dimensions.^{37,38}

AUTHOR DECLARATION

The authors have no conflicts to disclose.

CODE AVAILABILITY

Our code is available on github, https://github.com/schraderSimon/Rothe_L2O.

USE OF ARTIFICIAL INTELLIGENCE

While we are responsible for our own code, we made use of ChatGPT for debugging and code suggestions. In particular, ChatGPT has helped us deal with the rather convoluted syntax requirements of PyTorch, and to make the gradient of the Rothe error, which was written using Numpy, compatible with automatic differentiation. Furthermore, it has helped us write small utility functions regarding file management and visualization.

BIBLIOGRAPHY

- ¹D. J. Haxton and C. W. McCurdy, *Phys. Rev. A* **91**, 012509 (2015).
- ²D. J. Haxton, K. V. Lawler, and C. W. McCurdy, *Phys. Rev. A* **83**, 063416 (2011).
- ³T. Sato, H. Pathak, Y. Orimo, and K. L. Ishikawa, *J. Chem. Phys.* **148**, 051101 (2018).
- ⁴B. S. Ofstad, E. Aurbakken, Ø. S. Schøyen, H. E. Kristiansen, S. Kvaal, and T. B. Pedersen, *WIREs Comput. Mol. Sci.* **13**, e1666 (2023).
- ⁵K. Kaufmann, W. Baumeister, and M. Jungen, *J. Phys. B At. Mol. Opt. Phys.* **22**, 2223 (1989).
- ⁶E. Luppi and M. Head-Gordon, *J. Chem. Phys.* **139**, 164121 (2013).
- ⁷A. P. Woźniak, M. Lesiuk, M. Przybytek, D. K. Efimov, J. S. Prauzner-Bechcicki, M. Mandrysz, M. Ciappina, E. Pisanty, J. Zakrzewski, M. Lewenstein, and R. Moszyński, *J. Chem. Phys.* **154**, 094111 (2021).
- ⁸E. Coccia, B. Mussard, M. Labeye, J. Caillat, R. Taïeb, J. Toulouse, and E. Luppi, *Int. J. Quantum Chem.* **116**, 1120 (2016).
- ⁹S. Kvaal, C. Lasser, T. B. Pedersen, and L. Adamowicz, No need for a grid: Adaptive fully-flexible gaussians for the time-dependent Schrödinger equation (2023), arXiv:2207.00271 [quant-ph].
- ¹⁰S. E. Schrader, H. E. Kristiansen, T. B. Pedersen, and S. Kvaal, *J. Chem. Phys.* **161**, 044105 (2024).
- ¹¹S. E. Schrader, T. B. Pedersen, and S. Kvaal, *J. Chem. Phys.* **162**, 024109 (2025).
- ¹²M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. de Freitas, Learning to learn by gradient descent by gradient descent (2016), arXiv:1606.04474 [cs.NE].
- ¹³T. Chen, X. Chen, W. Chen, H. Heaton, J. Liu, Z. Wang, and W. Yin, Learning to Optimize: A Primer and A Benchmark (2021), arXiv:2103.12828 [math].
- ¹⁴Y.-C. Huang and H.-S. Goan, L2o-g[†]: Learning to optimize parameterized quantum circuits with fubini-study metric tensor (2024), arXiv:2407.14761 [quant-ph].
- ¹⁵I. Liao, R. R. Dangovski, J. N. Foerster, and M. Soljačić, Learning to optimize quasi-newton methods (2023), arXiv:2210.06171 [cs.LG].
- ¹⁶X. Chen, J. Liu, and W. Yin, *Science China Mathematics* **67**, 1191 (2024).
- ¹⁷L. Metz, C. D. Freeman, J. Harrison, N. Maheswaranathan, and J. Sohl-Dickstein, Practical tradeoffs between memory, compute, and performance in learned optimizers (2022), arXiv:2203.11860 [cs].
- ¹⁸J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, in *Proceedings of the 2019 Conference of the North* (Association for Computational Linguistics, Minneapolis, Minnesota, 2019) pp. 4171–4186.
- ¹⁹I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016) <http://www.deeplearningbook.org>.
- ²⁰A. Gu, K. Goel, A. Gupta, and C. Ré, arXiv preprint arXiv:2206.11893 (2022).
- ²¹A. Gu, K. Goel, and C. Ré, arXiv preprint arXiv:2111.00396 (2021).
- ²²A. Gu and T. Dao, arXiv preprint arXiv:2312.00752 (2023).
- ²³A. Daigavane, B. Ravindran, and G. Agarwal, Distill **10.23915/distill.00032** (2021), <https://distill.pub/2021/understanding-gnns>.
- ²⁴L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, *IEEE Transactions on Intelligent Transportation Systems* **21**, 3848–3858 (2020).
- ²⁵P. Deuffhard and M. Weiser, *Adaptive Numerical Solution of PDEs* (De Gruyter, Berlin, Boston, 2012).
- ²⁶G. H. Golub and V. Pereyra, *SIAM J. Numer. Anal.* **10**, 413 (1973).
- ²⁷I. Barth and C. Lasser, *J. Phys. B At. Mol. Opt. Phys.* **42**, 235101 (2009).
- ²⁸D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2014), arXiv:1412.6980 [cs.LG].
- ²⁹A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, Py-

- torch: An imperative style, high-performance deep learning library (2019), [arXiv:1912.01703 \[cs.LG\]](#).
- ³⁰M. Kuczmann, *Acta Technica Jaurinensis* **12**, pp. 130–147 (2019).
- ³¹J. T. Smith, A. Warrington, and S. W. Linderman, [arXiv preprint arXiv:2208.04933](#) (2022).
- ³²T. N. Kipf and M. Welling, *Semi-supervised classification with graph convolutional networks* (2017), [arXiv:1609.02907 \[cs.LG\]](#).
- ³³R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, Tune: A research platform for distributed model selection and training (2018).
- ³⁴J. Nocedal and S. J. Wright, *Numerical optimization* (Springer, New York, 1999).
- ³⁵E. Jones, T. Oliphant, P. Peterson, *et al.*, *SciPy: Open source scientific tools for Python* (2001–).
- ³⁶A. Chakrabarty, C. Danielson, and Y.-S. Wang, [arXiv preprint arXiv:2211.07768](#) (2022), accessed: 2025-05-24.
- ³⁷M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, *Deep sets* (2018), [arXiv:1703.06114 \[cs.LG\]](#).
- ³⁸J. Lee, Y. Lee, J. Kim, A. R. Kosiosek, S. Choi, and Y. W. Teh, *Set transformer: A framework for attention-based permutation-invariant neural networks* (2019), [arXiv:1810.00825 \[cs.LG\]](#).